HERMES

Long-Horizon Document Intelligence

Master Plan & Architecture

Confidential draft · 29 June 2026

> Project name: **Hermes**. Built on Baidu **Unlimited-OCR** + Claude (`claude-opus-4-8`).
>
> **Team:** backend — FastAPI orchestrator, OCR service, the `hermes` engine (one of us). Frontend — React + Vite (the other). See **Frontend brief** below for the API contract.

Context — why this exists

A new class of OCR model (DeepSeek-OCR → **Unlimited-OCR**) made something economically possible in 2026 that was impossible in 2025: **parsing arbitrarily long, messy, structured documents in a single forward pass at near-zero marginal cost, self-hosted.** Unlimited-OCR's **R-SWA (Reference Sliding Window Attention)** keeps memory and latency *flat* as output grows — so a 200-page document costs roughly what a 5-page one does, and reading order / tables / math survive. GPT-4o cannot do this: it costs a fortune in vision tokens, loses reading order across columns, and drops pages past its context limit.

The user wants to build a **unique, investor-impressive product** on this. This plan defines one **platform** (a parsing + reasoning engine) and a **phased vertical rollout** so the story is coherent, not scattershot. We lead with the cheapest-to-win wedge and expand toward the highest-value vertical.

Corrected technical record (must be right for investor DD):

- Our engine is **Unlimited-OCR** (3B MoE, DeepSeek-OCR lineage). Its honest, provable edge is **R-SWA → flat memory/latency on long docs**. Use that phrase, not "Layout-as-Thought" (unverified).
- Coordinates ARE available: prefix the prompt with `⊲grounding⊳` and the model interleaves text with `⊲det⊳span [x1,y1,x2,y2]` boxes — this powers cell-mapping and hover-overlay features.
- Do **not** claim "Qianfan-OCR is #1 on OmniDocBench" — **PaddleOCR-VL** tops it (v1.6 = 96.33). Unlimited-OCR ≈ 93.9, Qianfan ≈ 93.1. These are *three different models*.
- Handwriting/historical docs are **PaddleOCR-VL's** strength, not Unlimited-OCR's → vertical ③ requires a second model.

The unifying thesis (one engine, many verticals)

> "GPT-4o chokes on long, structured documents — it drops pages, scrambles columns, and burns vision tokens. We parse a 200-page legal bundle or financial disclosure into perfect structured data in one pass, for cents, then reason over it. One engine; we go vertical by vertical."

Platform = Parse → Structure → Reason → Deliver.

- **Parse:** Unlimited-OCR (printed/long) or PaddleOCR-VL (handwritten) → markdown + optional `⊲grounding⊳` coordinates.
- **Structure:** tables→grids, math→LaTeX, layout→reading order, boxes→cell/line coordinates.
- **Reason:** Claude (`claude-opus-4-8` , structured outputs) for extraction, verification, Q&A, citation, summarization.

- **Deliver:** the vertical-specific artifact (clean Markdown/DOCX, reconciled spreadsheet, searchable archive, API JSON).

The verticals (phased)

#	Vertical	Buyer	Engine	Risk	Revenue	Role
①	**Book-to-Markdown** (legal bundles, academic papers, textbooks)	Law firms, legal-tech, researchers, publishers	Unlimited-OCR	Low	Mid	**Wedge — build first**
②	**Financial audit / extraction** (disclosures, statements, nested tables)	Auditors, lenders, finance teams	Unlimited-OCR + grounding	High (accuracy/liability)	**High**	**Expansion vertical**
③	**Historical / handwritten archive** (registries, manuscripts)	Museums, universities, gov	**PaddleOCR-VL**	High (model + GTM)	Low (grant-funded)	**Prestige lighthouse — defer**
⓪	**Horizontal parsing API**	Developers / other apps	Unlimited-OCR	Med	Usage-based	**Underlying layer, sold later**

Strategy: Win ① (cheapest, truest to the tech, best demo) → expand into ② (same engine, higher-value docs, add a reconciliation/trust layer) → expose ⓪ as an API once the engine is hardened → use ③ as a credibility/PR lighthouse only if a grant or marquee archive partner appears.

Domain packs (the scalable unit + the real moat)

The platform is **horizontal**. Each "boring corporate" document type is added as a **Domain Pack**, not a rebuild:

- **Schema Pack** — the fields to extract (invoice vs. lab report vs. insurance policy vs. contract).
- **Rule Pack** — the checks to run (the audit/verification logic for that domain).

Packs to ship beyond ①: **Financial/Audit, Bank statements, Insurance (policy + claims), Clinical records, Legal contracts.** The accumulating library of rule packs + the correction data behind them is the moat — the OCR model is downloadable; the domain logic is not.

Domain Pack	"Audit" in one line	Buyer
Financial	Tick-and-tie: recompute totals, balance-sheet identity, cross-foot tables	Auditors, lenders
Bank	Reconcile balances, categorize txns, verify income, flag fraud	Lenders, fintechs
Insurance	Claim ≤ coverage limit, completeness, duplicate/fraud flags	Insurers, brokers
Clinical	Does the chart support the billed ICD/CPT codes? missing fields?	Hospitals, billers
Legal	Missing/conflicting clauses, obligations & deadlines, DD checklist	Law firms, in-house

How the audit / verification engine actually works

"Audit" is **not** the LLM passing judgment. It is mechanical tick-and-tie done at 100% coverage, with a human signing off the exceptions. Three deliberately-separated layers:

1. **Extraction (AI):** OCR + Claude (`messages.parse` + Pydantic) pull every number/field/clause into typed data. Fast; can misread → everything downstream carries a confidence score + a source citation.
2. **Verification (deterministic code — THIS is the audit):** pure arithmetic/logic, zero hallucination, fully auditable. E.g. line items = subtotal; subtotal + tax = total; Assets = Liabilities + Equity; "Total Revenue" p.12 = p.40; bank closing balance = balance-sheet Cash; cross-foot tables; required fields present; anomalies (duplicates, Benford's-law digit deviation, impossible dates).
3. **Explanation (AI) + human sign-off:** Claude writes the discrepancies in plain English, each with a **clickable citation to the exact `◁grounding▷` box on the exact page**; a human reviews flagged exceptions and signs.

Why it's trustworthy (the DD answer): the math is done by code, not the model. The LLM only *finds* and *explains*; the *verdict* is arithmetic, so it cannot hallucinate a number. Turns ~2 days of manual tick-and-tie into ~minutes of reviewing flagged exceptions.

Worked example — 50-page financial statement for a loan: parse all pages one-shot → structured statements → recompute every total + the balance-sheet identity → cross-check the bank statement's closing balance vs. Cash → output: "3 issues: (1) p.12 line items sum to ₦4.2M but subtotal says ₦4.5M [jump]; (2) balance-sheet Cash ₦8M ≠ bank statement ₦6.3M [jump]; (3) Invoice #0412 duplicated [jump]."

Honest risk: extraction errors propagate (a misread ₦4.5M→₦45M triggers a false flag). Mitigation = confidence scores + the grounding citation that lets a human jump to source and verify in one click. The system surfaces everything worth checking; it does not silently certify.

Domain Pack Specifications

Pitch lead = Financial Audit (richest deterministic rules → most defensible demo). **Revenue follow = Bank** (high volume, clear lender buyer). Legal/Clinical are more AI-classification-heavy (need more human review) → sequence them later. Each pack = a **Schema Pack** (fields to extract) + a **Rule Pack** (checks). Deterministic-heavy packs (Financial, Bank) are the most trustworthy; AI-heavy packs (Legal, Clinical) lean more on human sign-off.

1. Financial / Audit *(pitch lead)*

- **Schema:** entity, period, currency; Income Statement (revenue, COGS, gross profit, opex line items, operating income, interest, tax, net income); Balance Sheet (asset/liability/equity lines + totals); Cash Flow (operating/investing/financing, net change). Every figure carries value · page · bbox · confidence.
- **Rule Pack (deterministic):** gross profit = revenue − COGS; operating income = gross profit − Σ opex; balance-sheet identity (assets = liabilities + equity); cross-foot every total = Σ its lines; net change in cash ties to BS cash delta; retained-earnings roll-forward (RE_end = RE_begin + net income − dividends); same labeled figure equal across all pages; **cross-document** bank closing balance = BS cash. **Anomalies:** Benford's-law digit deviation, duplicate amounts, round-number clustering.

- **AI role:** locate/label statements in messy layouts, map synonyms ("Turnover"=Revenue), narrate discrepancies.
- **Output:** exception report + reconciled spreadsheet + audit trail. **Buyer:** external auditors, lenders, finance teams.

2. Bank Statements *(revenue follow)*

- **Schema:** holder, masked account no, bank, period, opening/closing balance, transactions [date, description, debit, credit, running balance], stated totals.
- **Rule Pack (deterministic): running-balance continuity** balance[i] = balance[i−1] ± txn[i] (the killer check — catches tampering *and* OCR errors); opening + Σ credits − Σ debits = closing; stated deposit/withdrawal totals reconcile; dates monotonic & within period. **Anomalies:** duplicate transactions, structuring (clusters of just-under-threshold deposits → AML flag).
- **AI role:** categorize transactions (income/rent/loan/transfer), detect recurring salary/income, classify counterparties.
- **Output:** verified monthly income, affordability/DTI inputs, tamper/fraud flags, categorized statement. **Buyer:** digital lenders, fintech underwriting, mortgage.

3. Insurance (Policy + Claims)

- **Schema:** Policy (insured, policy no, coverage types + limits + deductibles, exclusions, period, premium); Claim (claimant, claim no, date of loss, claimed amount, line items, supporting docs).
- **Rule Pack:** claim amount ≤ coverage limit − deductible (per peril); date of loss within policy period; claimed peril is covered (not in exclusions); claim line items sum = claimed total; required supporting docs present; duplicate-claim detection.
- **AI role:** parse policy clauses → structured coverage, match claim peril ↔ covered perils, explain coverage determination.
- **Output:** coverage-determination support, completeness checklist, fraud flags (adjuster signs). **Buyer:** insurers, TPAs, brokers.

4. Clinical / Medical-Coding

- **Schema:** de-identified patient, encounter, diagnoses (ICD-10), procedures (CPT/HCPCS), meds, labs/vitals, provider note, billed codes + charges.
- **Rule Pack: coding support** — each billed code justified by documentation in the chart (the core audit); medical-necessity linkage (procedure ↔ supporting diagnosis); E/M documentation completeness; NCCI unbundling/upcoding edits; duplicate billing; drug-interaction/dosage sanity.
- **AI role:** map free-text notes → candidate codes, judge whether documentation supports the billed code, surface under/over-coding.
- **Output:** coding-audit report (supported vs. unsupported codes), revenue-integrity & compliance findings. **Buyer:** hospitals, billing cos, payers.
- ⚠️ **PHI/HIPAA:** strict data handling — de-identify, prefer on-prem/self-host deployment. This pack carries the heaviest privacy architecture.

5. Legal Contracts

- **Schema:** parties, effective/term dates, governing law, obligations [party · obligation · due date], payment terms, key clauses (termination, liability cap, indemnity, IP, confidentiality, auto-renewal, assignment,

change-of-control), defined terms.

- **Rule Pack (checklist + logic, less arithmetic):** required-clause checklist present/absent (per playbook); conflicting terms (e.g. two governing-law clauses, payment mismatch across sections); cross-reference integrity (terms used-but-not-defined, section refs resolve); deviation from standard playbook positions (liability cap below threshold, missing indemnity).
- **AI role (heavier here):** clause classification, conflict detection, plain-English risk summary, redline suggestions; deterministic layer = checklist + date logic + reference integrity.
- **Output:** contract-review memo, obligations/deadlines matrix, missing-clause list, risk flags (lawyer signs). **Buyer:** law firms, in-house legal, M&A DD.

0. Book-to-Markdown (the wedge — no rule pack)

Clean structured export (Markdown/DOCX/LaTeX) + Q&A with page citations. No verification layer; pure parse quality + the cost demo. This is the cheapest pack to ship and the investor demo.

Investor narrative (deck outline)

1. **Hook / problem:** Knowledge work drowns in long, messy documents; the AI everyone uses (GPT-4o) silently fails on them — dropped pages, scrambled columns, runaway cost.
2. **Why now:** 2026 long-horizon OCR (R-SWA) collapsed the cost of parsing unlimited-length documents, self-hosted. The capability is 12 months old.
3. **Product:** Parse → Structure → Reason → Deliver. Live side-by-side demo (see below).
4. **Wedge → expansion:** Legal/academic now; financial next; API after. One engine.
5. **Market:** size ①+② (legal-tech + intelligent document processing IDP is a multi-$B, double-digit-growth market — pull current figures before pitching).
6. **Business model & unit economics:** cents-per-doc cost vs. per-page or seat pricing → high gross margin (below).
7. **Moat:** workflow depth + verification/trust layer + proprietary correction data flywheel + switching cost of the customer's corpus (below).
8. **Traction / GTM:** design-partner-led; embed into incumbent workflows.
9. **Team / why us / ask.**

Founder-credibility guardrails: name the exact model (Unlimited-OCR), the exact edge (R-SWA flat memory), the exact benchmark numbers (and that we know PaddleOCR-VL tops OmniDocBench). Investors trust founders who know precisely what they're standing on.

The killer demo (build this first — it IS the pitch)

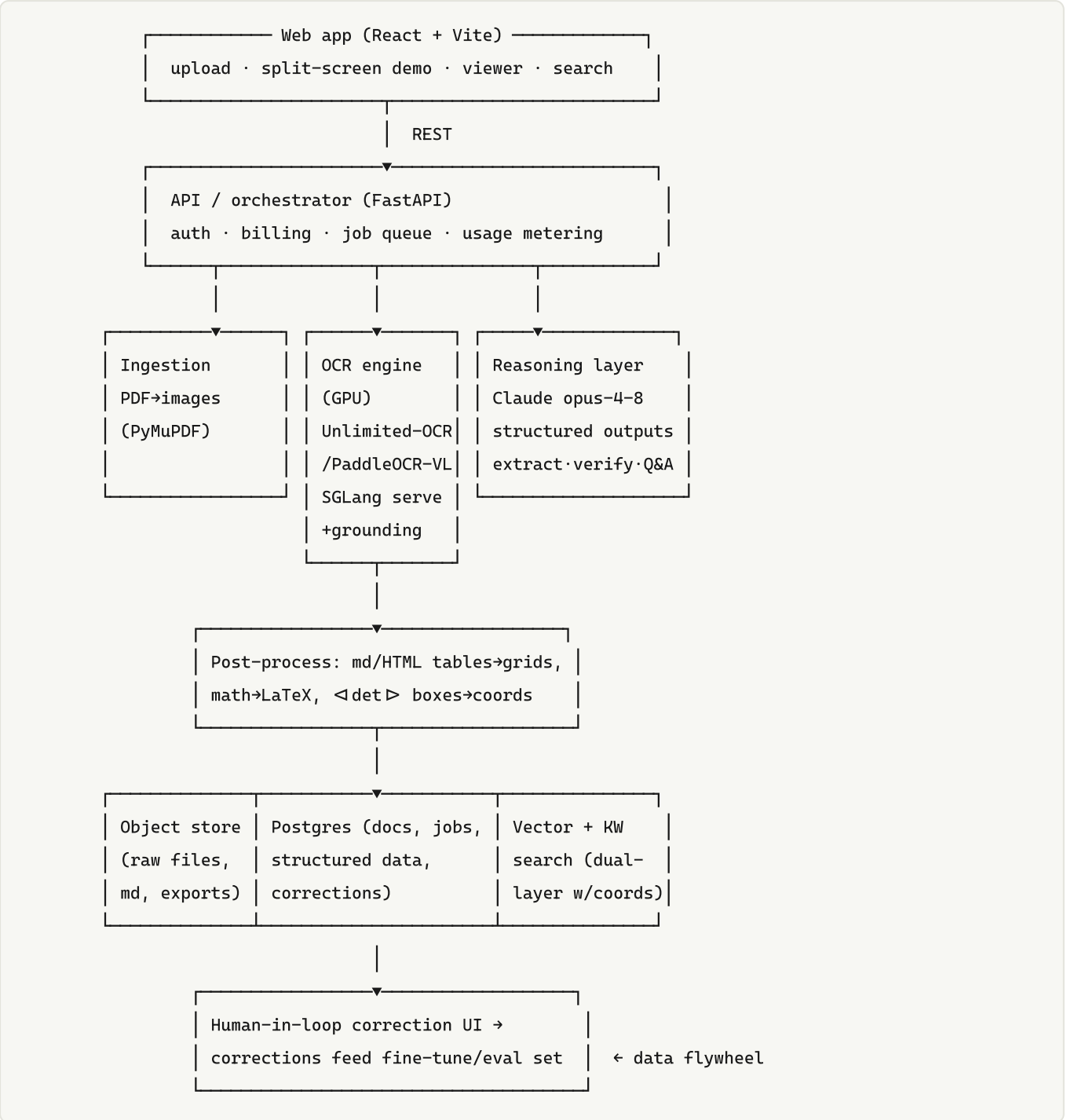
"The document that breaks GPT-4o." Split-screen, live:

- Left = GPT-4o / generic tool. Right = Hermes (Unlimited-OCR).
- Input: a deliberately brutal **50–200 page** doc — multi-column academic paper with equations, or a scanned legal bundle with page-spanning tables.
- Show a **scoreboard** updating live: pages successfully parsed, reading-order errors, tables preserved, **$ cost**, wall-clock time.

- Outcome: GPT-4o drops pages / scrambles columns / costs $X; Hermes returns clean Markdown (+ LaTeX math, + tables) in one pass for ~$0.0X.
- Finale: ask a question over the parsed doc with citations to exact pages (Claude + grounding coordinates) — proving Parse→Reason, not just OCR.

This single artifact carries slides 3, 6, and 7. Build the benchmark harness so the numbers are reproducible on demand.

Technical architecture



Key decisions:

- **OCR serving:** MVP = Transformers `model.infer()` (the notebook). Prod = **SGLang** server (`--attention-backend fa3`, needs Hopper/H100) for throughput + the OpenAI-compatible streaming API. Host on Modal / RunPod / Lambda. T4 (Colab/Kaggle) is demo-only.
- **Model router:** printed/long → Unlimited-OCR; handwriting-heavy (vertical ③) → PaddleOCR-VL. One interface, two backends.
- **Grounding:** enable `⊲grounding⊳` when the vertical needs coordinates (cell mapping in ②, hover-overlay in ③); skip it for plain ① throughput.
- **Reasoning:** `client.messages.parse()` + Pydantic schemas (already prototyped in the notebook §8) for extraction/verification; citations via grounding boxes.
- **Trust layer (the real product in ②):** confidence flags, reconciliation rules (e.g. line items must sum to stated total), human-in-loop correction, full audit trail.

MVP Deployment Architecture (decided)

Goal: a public link people can open and test; scale-to-zero cost; minimal ops. No Colab (it's a notebook runtime — no stable URL, disconnects). Three managed tiers + a swappable GPU service.

Confirmed stack: Frontend = **React + Vite + TypeScript** (Tailwind + shadcn/ui for a product-grade look, TanStack Query for job-polling). Backend = **FastAPI** (Python is mandatory — OCR model, `hermes`, Anthropic SDK, PyMuPDF all live there). Access = **open, no login** for the MVP.

```
React (Vercel) ──HTTPS──▶ FastAPI orchestrator (CPU PaaS) ──HTTPS──▶ OCR service (GPU)
  UI + polling              jobs · Claude · hermes engine              Unlimited-OCR only
```

Tiers:

- **Frontend — React on Vercel** (Netlify equiv). Static; free HTTPS. Upload → poll job → render result (parsed markdown + audit findings with click-to-source). Hero screen = the split-screen "breaks GPT-4o" demo.
- **Orchestrator — FastAPI on a CPU PaaS** (Render / Railway / Fly.io). Cheap, HTTPS included, **no GPU**. Owns: upload handling, async job state, PDF→images (PyMuPDF), Claude extraction (`messages.parse`), the `hermes` rule engine, and the result API. **The Anthropic key lives only here, never in React.**
- **OCR service — Unlimited-OCR behind ONE HTTP contract**, deployable two ways without changing the orchestrator:
  - **Dev / now:** the existing GPU box — run the OCR service with uvicorn; expose a stable URL via Cloudflare Tunnel or Tailscale.
  - **Deployed MVP link: Modal** serverless GPU — HTTPS endpoint, scale-to-zero. Same contract, swap the URL.
- **Storage:** Postgres (jobs/results — Render/Railway managed) + S3-compatible object store (Cloudflare R2 / Supabase) for uploads. Local disk is fine for the very first cut.

The OCR service contract (the integration seam that makes the GPU tier swappable): `POST /ocr` (image or page set, + `grounding: bool`) → `{ markdown, boxes: [{text, bbox, page}], pages, latency_ms }`.

Async job flow (don't block — OCR on a big PDF takes minutes):

1. `POST /documents` (multipart) → store file, create job `queued`, start background task, return `{job_id}`.

2. Background: PDF→images → OCR service → Claude extraction → `hermes` audit → store result, status `done`.
3. `GET /documents/{job_id}` → status + result; React polls.

Day-one gotchas baked in: CORS for the Vercel origin; HTTPS everywhere (managed hosts give it free); Modal cold-start → show a "warming up the engine" state; upload size limits; all secrets server-side only. **Open access cost guard (required since there's no login):** a per-IP/session rate limit + a global daily spend cap, so a random visitor or abuse can't run up the GPU/Claude bill.

Repo layout (monorepo):

- `apps/web` — React (Vercel)
- `apps/api` — FastAPI orchestrator (jobs, Claude, imports `hermes/`)
- `services/ocr` — Unlimited-OCR HTTP service (one Modal app entry + one local-uvicorn entry, same handler)
- `hermes/` — the verification engine (already built: `core.py`, `financial.py`), imported by `apps/api`

Verify the MVP end-to-end: deploy the three tiers → open the Vercel link → upload a planted-error statement (like `hermes/demo_financial.py`'s sample) → confirm the job completes and the audit report renders with click-to-source citations. Then repeat against the *real* OCR path (a scanned PDF) once the OCR service is wired.

Frontend brief (for the partner — React + Vite)

The backend (`apps/api`, already built) exposes a tiny REST surface. Build against this contract; all OCR/Claude detail is hidden behind it.

Base URL: `VITE_API_URL` env var. Local dev: `http://localhost:8000`.

Endpoints:

- `GET /health` → `{ ok, mock_ocr, mock_extraction, model }` — drives a status badge.
- `POST /documents` — multipart form, field name **`file`** → `{ job_id, status }`. Errors: `400` empty file; `429` rate-limited (`detail` = message to display).
- `GET /documents/{job_id}` → `{ job_id, status, filename, result, error }`. `status` ∈ `queued | processing | done | error`; `result` is `null` until `done`.

`result` shape (when `status == "done"`):

```
{
  "filename": "statement.pdf",
  "pages": 1,
  "markdown": "# … parsed document … ",
  "entity": "Acme Trading Ltd",
  "period": "FY2025",
  "currency": "₦",
  "summary": { "checks": 10, "passed": 6, "flagged": 4 },
  "findings": [
    { "rule": "balance_sheet_identity", "passed": false, "severity": "error",
      "message": "liabilities + equity = ₦12.8M but total assets = ₦15M … ",
      "expected": 12800000, "actual": 15000000,
      "citations": ["Total assets (p.28)", "Total liabilities (p.29)", "Total equity (p.29)"] }
  ],
  "report_text": "plain-text version of the report"
}
```

Screens (MVP):

1. **Upload** — drag/drop a PDF or image → `POST /documents`.
2. **Processing** — poll `GET /documents/{job_id}` with TanStack Query (~1.5s interval) until `done` / `error`; show a "warming up the engine" state (GPU cold-start).
3. **Results** — two panes: left = rendered `markdown` (markdown renderer; KaTeX for math later); right = **findings list**, flagged first, colored by `severity` (`error` / `warning` / `info`), each showing `message` + `citations`; summary counts on top; a "download report" button (`report_text`).
4. **(later)** the split-screen "breaks GPT-4o" hero demo.

Notes: `citations` are human-readable strings now; later they carry page+bbox for click-to-jump. Handle `429` by surfacing `detail` (the open-access rate limit).

Business model & unit economics

- **Pricing:** per-page or per-document tiers + SaaS seats for the workflow app; usage-based for the ⓪ API. Anchor against GPT-4o vision cost and legacy per-page OCR pricing — your marginal cost is a self-hosted GPU-second (cents), so **gross margin is high**.
- **Land:** cheap self-serve for ① (researchers, solo lawyers). **Expand:** seat + volume contracts for ② (audit/finance teams). API ⓪ as a third revenue line.
- Quantify the demo's cost delta and put it on a slide — it's the most persuasive number you have.

The moat problem (address head-on — investors WILL probe it)

Unlimited-OCR is open-source (MIT). The model is not the moat. Defensibility comes from:

1. **Workflow depth** — ship the *deliverable* (filed, reconciled, searchable), not raw text. Domain logic per vertical.
2. **Verification / trust layer** — confidence, reconciliation, human-in-loop, audit trail. This is what regulated buyers actually pay for.
3. **Proprietary correction data flywheel** — every human correction (captured with permission) builds a fine-tune + eval set on hard, domain-specific document types competitors can't replicate without your volume.
4. **Switching cost** — once a firm's corpus lives in your search/system, leaving is painful.
5. **Distribution** — embed into the tools these users already live in (e.g. legal practice-management, ERPs).

Build roadmap (off the existing notebook)

The notebook `unlimited_ocr_demo.ipynb` already proves: OCR (§5), tables→Excel (§6), Gradio UI (§7), Claude structured extraction (§8–9). Promote it to a product:

- **Phase 0 — De-risk (week 1):** Confirm Unlimited-OCR runs (Colab T4), then validate `◁grounding▷` output and a 50-page one-shot parse on a real GPU. **Gate:** if grounding/long-doc works, proceed; if not, re-scope.
- **Phase 1 — The demo (weeks 1–3):** Build the split-screen benchmark harness (Hermes vs GPT-4o: pages, reading-order errors, cost, latency). This is the investor artifact and the first thing to show.
- **Phase 2 — Vertical ① MVP (weeks 3–8):** Deploy the three tiers (React/Vercel + FastAPI/CPU PaaS + Unlimited-OCR on Modal serverless; dev against the existing GPU box). Upload → OCR → clean Markdown/DOCX + Claude Q&A-with-citations + the `hermes` audit. Public test link. One or two design-partner lawyers/researchers/finance teams.
- **Phase 3 — Vertical ② layer (post-MVP):** Add grounding-based cell mapping + reconciliation/verification + human-in-loop correction UI. Sell to a lender/audit design partner.
- **Phase 4 — API ⓪ + flywheel:** Expose the parsing API; wire corrections into a fine-tune/eval pipeline.
- ③ deferred unless a grant/marquee archive partner appears (and budget the PaddleOCR-VL swap).

Naming

Chosen (for now): Hermes — messenger/courier of documents. Verify domain + trademark availability before committing externally; fallbacks if taken: Palimpsest, Folio, Marginalia, Codex, Scripta.

Risks

- **Kernel/GPU risk:** Unlimited-OCR's `trust_remote_code` kernels are unverified on older GPUs (T4). Prod needs modern GPUs; SGLang's fa3 backend needs Hopper. Validate early (Phase 0).
- **Accuracy/liability (②):** wrong financial numbers = lawsuits. Never market "guaranteed" extraction; sell verification + human-in-loop.
- **Commoditization:** the model is open; without the workflow/data moat this is a thin wrapper. The flywheel is non-optional.

- **Handwriting (③):** needs a different model and a different (slow, grant-funded) GTM — don't let it pull focus from ①/②.
- **Incumbents in ②:** Ocrolus, Rossum, etc. — differentiate on long-doc one-shot parsing + price, not on being "another IDP."

Verification (how we'll prove it works)

1. **Engine validation (Phase 0):** on a real GPU, run a 50–200 page PDF through Unlimited-OCR one-shot; confirm no dropped pages, correct reading order, tables→markdown, math→LaTeX, and `⊲grounding⊳` returns `[x1,y1,x2,y2]` boxes.
2. **Benchmark harness (Phase 1):** same doc through GPT-4o and Hermes; record pages parsed, reading-order errors, tables preserved, **$ cost**, latency. These numbers are the pitch — they must be reproducible live.
3. **End-to-end MVP (Phase 2):** upload → parse → Markdown/DOCX export → Claude Q&A with page citations, verified manually on 5–10 real documents from a design partner.
4. **Trust layer (Phase 3):** on financial docs, confirm reconciliation rules catch injected errors (e.g. line items not summing to total) and that corrections persist to the dataset.

Open assumptions to confirm with the user (non-blocking)

- Geography/first market (Nigeria/Africa vs global) — affects ① buyer (which legal/academic market) and pricing.
- Whether to register the ⓪ API ambition in the *first* investor story or keep it as a later reveal.
- Access to a design partner in ① or ② (warm intro changes the GTM section).